SOUND REACTIVE SYSTEM

Curiosity Kickoff

- What happens when you clap?
- What happens when sound is detected?
- Can we control devices using sound?

Concept Connection

A Sound Reactive System uses a sound sensor to detect audio signals and control LEDs accordingly. It converts sound into electrical signals, making lights glow or blink based on intensity. This system demonstrates how sound can be used to create interactive lighting effects in electronic applications.

1.Watt SMD LED

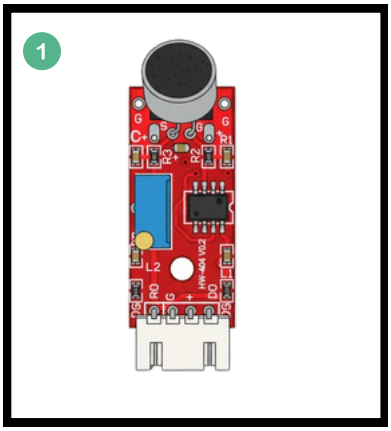
I'm flat and powerful. Just one of me can light up your whole table like a torch.I don't like wasting energy, so I shine super bright without getting too hot,You've already met me — I live in your mobile flashlight!

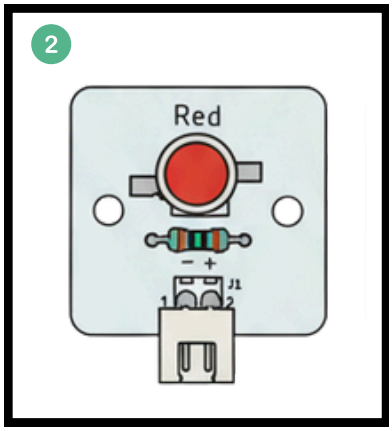
2. Sound Sensor

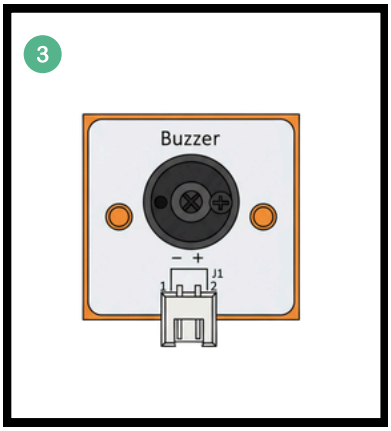
Has a small microphone inside that converts sound waves (claps, voices, knocks) into electrical signals. The circuit then compares the loudness to a set level and sends a digital or analog signal to Arduino.

3.Buzzer

A sound-making device that works when electricity makes a thin metal plate vibrate rapidly inside. The vibration creates buzzing or beeping sounds. Some buzzers can even change frequency to play simple tunes or melodies.







Beginner: Sound Trigger

Class Explanation

This project reads sound intensity using a sensor connected to an Arduino analog pin. The microcontroller continuously measures sound levels and displays values on the Serial Monitor. It helps monitor noise levels in real time, making it useful for basic sound detection, environmental monitoring, or simple audio-reactive projects.

Materials Required

- Arduino with attached shield
- 1 × Sound sensor
- RMC wires
- USB cable

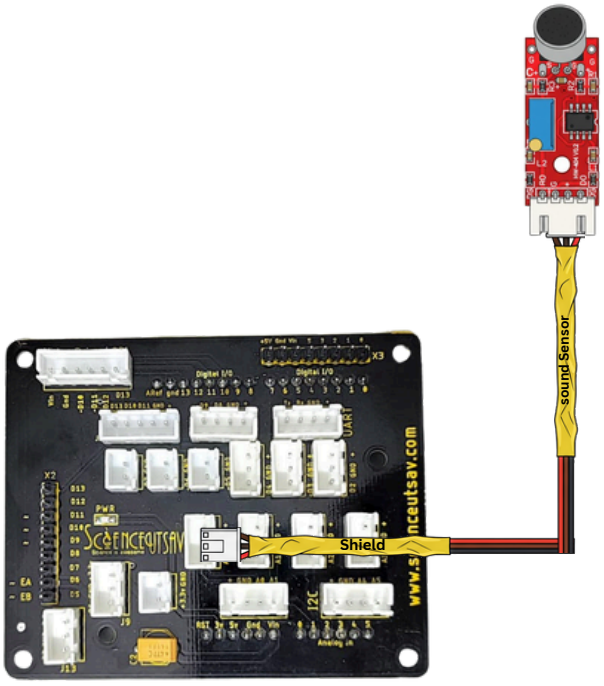
How to Connect

Use RMC wires to connect :

- 1 × Sound sensor→ Arduino pin A0 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define SOUND_PIN A0   // Analog output pin from sound sensor

void setup() {
 Serial.begin(9600); // Start the Serial Monitor
}

void loop() {
 int soundValue = analogRead(SOUND_PIN); // Read the sound sensor value (0-1023)

 Serial.print("Sound value: ");
 Serial.println(soundValue); // Print the sound value

 delay(100); // Small delay to smooth the readings
}
```

Code Explanation

- In this Arduino program, we define the sound sensor connected to analog pin A0 (SOUND_PIN).
- The setup() function runs once when the Arduino starts. It begins serial communication using Serial.begin(9600) so we can view the sound values on the Serial Monitor.
- The loop() function runs continuously. It reads the sound level using analogRead(), which gives values from 0 to 1023, and stores it in soundValue.
- Then, the value is displayed on the Serial Monitor using Serial.print() and Serial.println().
- A small delay() is added to make the readings smooth.

So overall, the Arduino continuously reads and displays the sound intensity from the sensor.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Sound Trigger Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define SOUND_PIN A0    // Sound sensor on A0
#define BUZZER_PIN 5    // Buzzer on D5

void setup() {
  Serial.begin(9600);
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  int soundValue = analogRead(SOUND_PIN); // Read sound (0–1023)

  Serial.print("Sound value: ");
  Serial.println(soundValue);

  // If sound is loud, turn buzzer ON
  if (soundValue > 500) {
    digitalWrite(BUZZER_PIN, HIGH); // Buzzer ON
  } else {
    digitalWrite(BUZZER_PIN, LOW);  // Buzzer OFF
  }

  delay(100);
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Sound Trigger Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Sound Reactive Alert

Class Explanation

In this class, students learn A Sound Reactive Alert system uses a sound sensor to detect noise levels and trigger an alert through LEDs or alarms. It converts sound into electrical signals and activates the output when sound exceeds a set level, helping in security, monitoring, and noise detection applications.

Materials Required

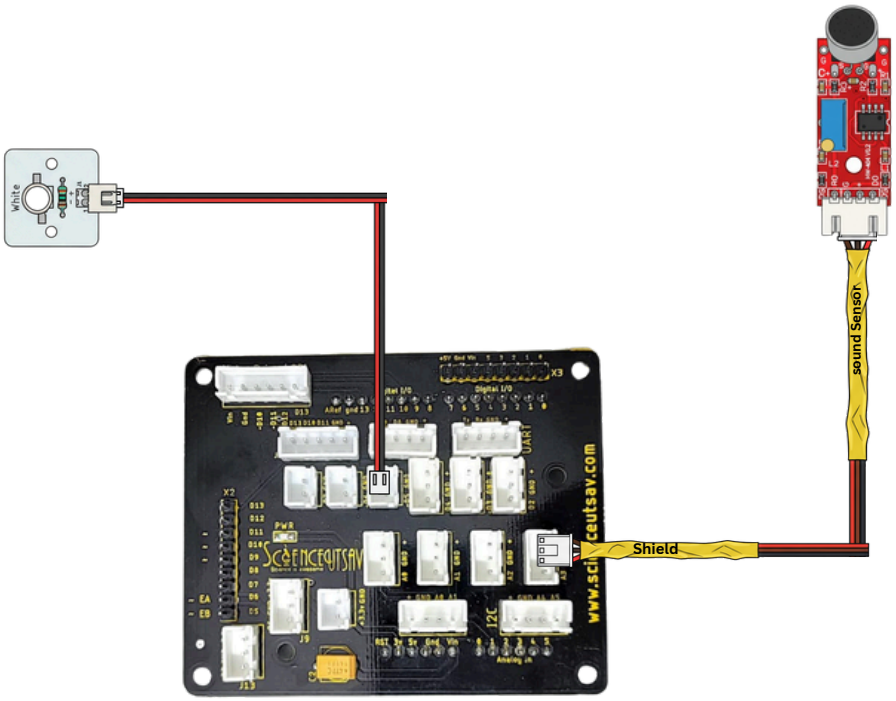
- Arduino with attached shield
- 1 × LDR
- 1 × white 1 Watt SMD LED on breakout board
- 1 × Red 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

How to Connect

- Use RMC wires to connect :
- 1 × LDR→ Arduino pin D2 and GND
- 1 × white 1 Watt SMD LED → Arduino pin D6 and GND
- 1 × RED 1 Watt SMD LED→ Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define SOUND_PIN A3 // Analog output pin from sound sensor
#define LED_PIN 5   // LED connected to D5

void setup() {
 Serial.begin(9600); // Start the Serial Monitor
 pinMode(LED_PIN, OUTPUT);
}

void loop() {
 int soundValue = analogRead(SOUND_PIN); // Read the sound sensor value (0-1023)
 Serial.print("Sound value: ");
 Serial.println(soundValue);      // Print the sound value

 int brightness = map(soundValue, 0, 1023, 0, 255); // Map to PWM range
 analogWrite(LED_PIN, brightness);         // Adjust LED brightness

 delay(100); // Small delay to smooth the readings
}
```

Code Explanation

- In this Arduino program, the sound sensor is connected to analog pin A3 (SOUND_PIN) and the LED is connected to pin 5 (LED_PIN).
- The setup() function runs once when the Arduino starts. It begins serial communication using Serial.begin(9600) and sets the LED pin as an OUTPUT.
- The loop() function runs continuously. It reads the sound level using analogRead() and stores it in soundValue, then displays it on the Serial Monitor.
- Next, the map() function converts the sound value (0–1023) into a range of 0–255. Using analogWrite(), the LED brightness is controlled based on the sound intensity.
- A small delay() is added for smooth readings.

So overall, the Arduino reads the sound level and adjusts the LED brightness accordingly.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Sound Reactive Alert System with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define SOUND_PIN A3 // Analog output pin from sound sensor
#define LED_PIN1 5   // LED1 connected to D5
#define LED_PIN2 6   // LED2 connected to D6

void setup() {
 Serial.begin(9600); // Start the Serial Monitor
 pinMode(LED_PIN1, OUTPUT);
 pinMode(LED_PIN2, OUTPUT);
}

void loop() {
 int soundValue = analogRead(SOUND_PIN); // Read the sound sensor value (0-1023)
 Serial.print("Sound value: ");
 Serial.println(soundValue);      // Print the sound value

 int brightness = map(soundValue, 0, 1023, 0, 255); // Map to PWM range

 analogWrite(LED_PIN1, brightness); // Control LED1 brightness
 analogWrite(LED_PIN2, brightness); // Control LED2 brightness

 delay(100); // Small delay to smooth the readings
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Sound Reactive Alert System with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: Sound Reactive Lighting

Class Explanation

In this class, students learn the A Sound Reactive Lighting system uses a sound sensor to detect audio signals and control LEDs accordingly. It converts sound into electrical signals, making lights glow or blink based on sound intensity, creating dynamic and interactive lighting effects for various applications.

Materials Required

- Arduino with attached shield
- 1 × Sound sensor
- 3 × White 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

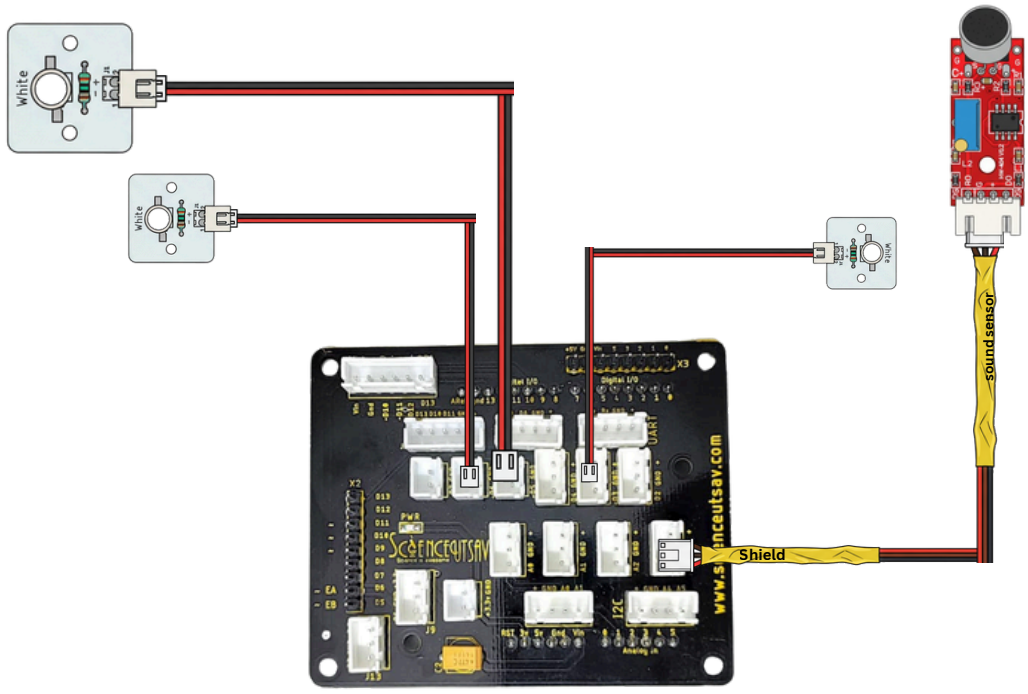
How to Connect

Use RMC wires to connect :

- 1 × Sound sensor → Arduino pin A3 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D5 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D5 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define SOUND_PIN A3  // Sound sensor on A3

#define LED1_PIN 5    // LED1 on D5
#define LED2_PIN 6    // LED2 on D6
#define LED3_PIN 3    // LED3 on D3

void setup() {
  Serial.begin(9600);

  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(LED3_PIN, OUTPUT);
}

void loop() {
  int soundValue = analogRead(SOUND_PIN);

  Serial.print("Sound value: ");
  Serial.println(soundValue);

  int brightness = map(soundValue, 0, 1023, 0, 255);

  // Apply same brightness to all LEDs
  analogWrite(LED1_PIN, brightness);
  analogWrite(LED2_PIN, brightness);
  analogWrite(LED3_PIN, brightness);

  delay(100);
}
```

Code Explanation

- In this Arduino program, a sound sensor is connected to analog pin A3 (SOUND_PIN), and three LEDs are connected to pins 5, 6, and 3 (LED1_PIN, LED2_PIN, LED3_PIN).
- The setup() function runs once when the Arduino starts. It begins serial communication using Serial.begin(9600) and sets all three LED pins as OUTPUT.
- The loop() function runs continuously. It reads the sound level using analogRead() and stores it in soundValue, then prints it on the Serial Monitor.
- Next, the map() function converts the sound value (0–1023) into a range of 0–255. Using analogWrite(), the same brightness value is applied to all three LEDs.
- A small delay() is added for smooth readings.

So overall, all three LEDs glow and change brightness together based on the sound intensity.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Sound Reactive Lighting but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define SOUND_PIN A3   // Analog output pin from sound sensor
#define LED_PIN 6      // LED moved to D6 (PWM pin)
#define BUZZER_PIN 5   // Buzzer connected to D5

void setup() {
  Serial.begin(9600);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  int soundValue = analogRead(SOUND_PIN); // Read sound (0–1023)

  Serial.print("Sound value: ");
  Serial.println(soundValue);

  int brightness = map(soundValue, 0, 1023, 0, 255);
  analogWrite(LED_PIN, brightness); // LED brightness based on sound

  // Buzzer logic (turn ON if sound is loud)
  if (soundValue > 500) {
    digitalWrite(BUZZER_PIN, HIGH); // Buzzer ON
  } else {
    digitalWrite(BUZZER_PIN, LOW);  // Buzzer OFF
  }

  delay(100);
}
```

Obective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Sound Reactive Lighting how the code connects with the circuit behavior
how to upload and test a simple Arduino sketch

Expert: Acoustic Logic System

Class Explanation

An Acoustic Logic System combines sound detection with logic circuits to control outputs like LEDs or alarms. It uses a sound sensor to capture audio signals and processes them through logic operations, enabling devices to respond intelligently to different sound conditions.

Materials Required

- Arduino with attached shield
- 1 × sound sensor
- 1 × 7 segment display
- RMC wires
- USB cable

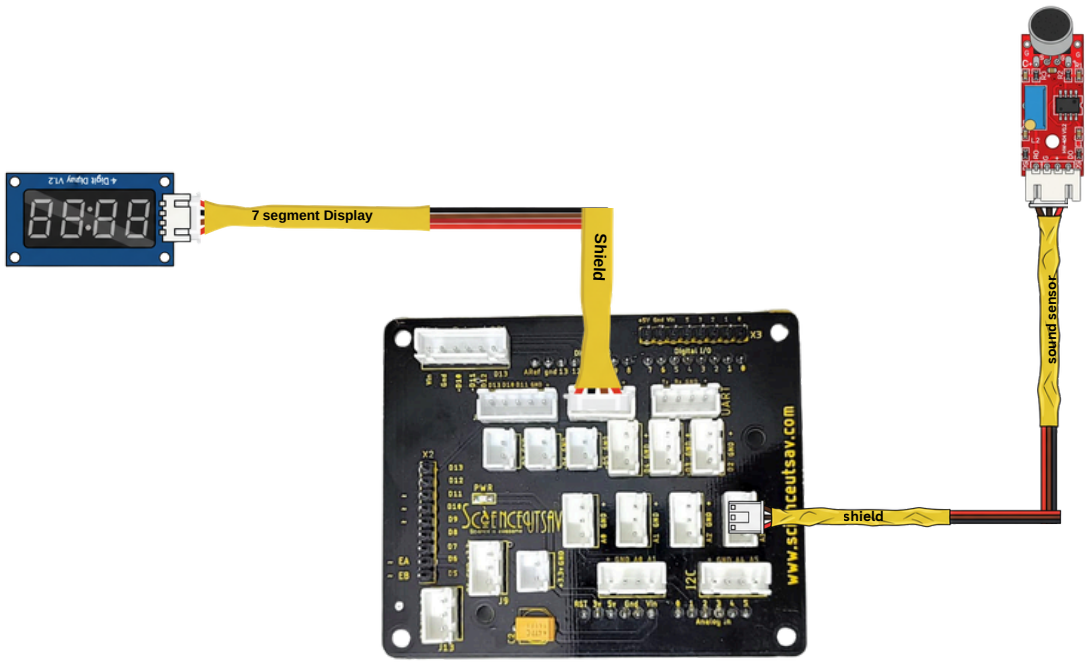
How to Connect

Use RMC wires to connect :

- 1 × Sound sensor → Arduino pin A3 and GND
- 1 × 7 segment display → Arduino pin D8 D9 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
// Sound Level → TM1637 dB Display
// Sensor: analog output on A3
// Display: TM1637 4-digit (CLK=D8, DIO=D9)
// Board: Arduino UNO (or compatible)

#include <Arduino.h>
#include <TM1637Display.h>
#include <math.h>

#define SOUND_PIN A3 // Analog output from sound sensor
#define CLK_PIN 8  // TM1637 CLK
#define DIO_PIN 9  // TM1637 DIO

TM1637Display display(CLK_PIN, DIO_PIN);

// ---- Tuning knobs ----
const unsigned long SAMPLE_MS = 50; // sampling window
const float VCC_VOLTS = 5.0;     // your board supply
const float REF_VOLTS = 0.010f;   // ~10 mV reference for dB calc
const int CAL_OFFSET = 30;     // shift reading up/down after log calc

void setup() {
 Serial.begin(9600);
 display.setBrightness(7);     // 0..7
 display.clear();
}

int readDb() {
 unsigned long start = millis();
 int signalMax = 0;
 int signalMin = 1023;

 // Capture peak-to-peak over a short window
 while (millis() - start < SAMPLE_MS) {
  int sample = analogRead(SOUND_PIN);
  if (sample > signalMax) signalMax = sample;
  if (sample < signalMin) signalMin = sample;
 }

 int p2p = signalMax - signalMin;       // peak-to-peak counts
 float volts = p2p * (VCC_VOLTS / 1023.0f);  // convert to volts
```

```
 if (volts < 0.001f) volts = 0.001f;

 float dB = 20.0f * log10f(volts / REF_VOLTS);
 dB += CAL_OFFSET;

 if (dB < 0) dB = 0;
 if (dB > 99) dB = 99;

 return (int)(dB + 0.5f);
}

void loop() {
 int dB = readDb();

 Serial.print("Approx dB: ");
 Serial.println(dB);

 display.showNumberDec(dB, true);
 delay(50);
}
```

Code Explanation

- In this Arduino program, a sound sensor is connected to analog pin A3 (SOUND_PIN) and a TM1637 4-digit display is connected to pins 8 (CLK_PIN) and 9 (DIO_PIN). The display is used to show the sound level in approximate decibels (dB).

- The setup() function runs once when the Arduino starts. It begins serial communication using Serial.begin(9600) and initializes the display using setBrightness() and clear().

- A separate function called readDb() is used to calculate the sound level. It reads multiple samples from the sound sensor for a short time and finds the maximum and minimum values. The difference (peak-to-peak value) is converted into voltage.

- Then, using a logarithmic formula, the voltage is converted into an approximate dB value. A calibration offset is added, and the value is limited between 0 and 99.

- The loop() function runs continuously. It calls readDb() to get the sound level, prints it on the Serial Monitor, and displays it on the TM1637 display using showNumberDec().

So overall, the Arduino measures sound intensity, converts it into dB, and shows it on a digital display.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Acoustic Logic System but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <Arduino.h>
#include <TM1637Display.h>
#include <math.h>

#define SOUND_PIN A3
#define CLK_PIN   8
#define DIO_PIN   9

#define LED_PIN    6   // LED on D6
#define BUZZER_PIN 5   // Buzzer on D5

TM1637Display display(CLK_PIN, DIO_PIN);

// ---- Tuning knobs ----
const unsigned long SAMPLE_MS = 50;
const float VCC_VOLTS = 5.0;
const float REF_VOLTS = 0.010f;
const int   CAL_OFFSET = 30;

// Threshold for alert
const int SOUND_THRESHOLD = 60; // dB level for LED & buzzer

void setup() {
  Serial.begin(9600);

  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  display.setBrightness(7);
  display.clear();
}

int readDb() {
  unsigned long start = millis();
  int signalMax = 0;
  int signalMin = 1023;
```

```
 while (millis() - start < SAMPLE_MS) {
 int sample = analogRead(SOUND_PIN);
 if (sample > signalMax) signalMax = sample;
 if (sample < signalMin) signalMin = sample;
 }

 int p2p = signalMax - signalMin;
 float volts = p2p * (VCC_VOLTS / 1023.0f);

 if (volts < 0.001f) volts = 0.001f;

 float dB = 20.0f * log10f(volts / REF_VOLTS);
 dB += CAL_OFFSET;

 if (dB < 0) dB = 0;
 if (dB > 99) dB = 99;

 return (int)(dB + 0.5f);
}

void loop() {
 int dB = readDb();

 Serial.print("Approx dB: ");
 Serial.println(dB);

 display.showNumberDec(dB, true);

 // LED + Buzzer control
 if (dB >= SOUND_THRESHOLD) {
 digitalWrite(LED_PIN, HIGH); // LED ON
 digitalWrite(BUZZER_PIN, HIGH); // Buzzer ON
 } else {
 digitalWrite(LED_PIN, LOW); // LED OFF
 digitalWrite(BUZZER_PIN, LOW); // Buzzer OFF
 }

 delay(50);
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn:
the basic idea of Acoustic Logic System how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes
After completing this session, students should be able to:

- Understand how sound sensors detect audio signals and control electronic outputs based on different sound levels and conditions.
- Learn how sound can trigger alerts using sensors, activating LEDs or alarms when sound exceeds a certain threshold level.
- Understand how sound signals are used to control lighting effects, making LEDs glow or blink according to sound intensity.
- Learn how sound input is processed using logic circuits to control outputs, enabling intelligent response to different sound conditions.

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output